4th class–

1- How python works:

i- Compilation process:

While running a Python script (.py), the Python compiler first checks your code for syntax errors. If the code is valid, it translates the source code into an intermediate format called Bytecode.

i-What is Bytecode: It is a low-level, platform-independent set of instructions. It isn't machine code that CPU understands.Only Pvm only understand bycode instructions. it is specific to Python.

ii-It is often stored in a folder named __pycache__ with a .pyc extension.

ii. Interpretation

The Bytecode is then sent to the Python Virtual Machine (PVM). The PVM contains the Interpreter,

i-The Interpreter reads the Bytecode line-by-line.

ii-It converts each line into Machine Code (binary 0s and 1s) that your specific computer hardware can execute.

2. Compiler

What it does: It translates the entire source code into machine code all at once before the program runs.

3. Interpreter

What it does: It translates and executes the code line-by-line during runtime.

–5th class–

1. Data Types in Python

Data types 5 types such as:

- `Int`: numbers (e.g., `10`, `-5`).
- `Float`: Decimal numbers (e.g., `10.5`, `3.14`).
- String: in quotes (e.g., `"Hello"`).
- `bool`: either `True` or `False`.
- None: It indicates that a variable exists but doesn't have a value yet.

---

2. Operators in Python

- + (Addition), - (Subtraction), * (Multiplication), / (Division)
- // (Floor Division): Returns the whole number after division (e.g., `7 // 2 = 3`).
- % (Modulus): Returns the remainder (e.g., `7 % 2 = 1`).
- (Exponent): Power (e.g., `2^3 = 8`).

Another Comparison Operators

- == (Equal), != (Not Equal)
- > (Greater than), < (Less than)
- >= (Greater than or equal), <= (Less than or equal)

C. Logical Operators

- `and`: Returns `True` if both statements are true.
- `or`: Returns `True` if one statement is true.
- `not`: Reverses the result (e.g., `not(True)` is `False`)

D. Assignment Operators

Used to assign values to variables.

- = (Assignment), += (Add and assign), -= (Subtract and assign).

—--6th class—-

1. Input & Type Conversion

By default, the input() function always takes data as a String.

- Concatenation: Using + with inputs will join them as text (e.g., "10" + "20" results in "1020").
- Type Casting: To perform math, need to convert the string to an integer using int().
    - *Example:* result = int(first) + int(second)

2. String Formatting Approaches

several ways to combine strings and variables:

- f-strings : The most modern and readable way.
    - *Syntax:* f"Hello {name}"
- .join() Method: *Syntax:* " ".join([first_name, last_name])
- .format() Method: Uses placeholders {} to insert values.
    - *Syntax:* "{} {}".format(a, b),if i extend 1 variable,then will be needed {} more extra one.
- String Multiplication: Make repetition a string multiple times using *.
    - *Example:* "sorry " * 1000

3. Useful String Methods

Python built-in functions:

- .upper() / .lower(): Converts text to all capital or all small letters.
- .title(): Capitalizes the first letter of every word.
- .capitalize(): Capitalizes only the very first letter of the string.
- .swapcase(): Flips cases (Lower to right , Upper to Lower).
- .count("x"): Counts how many times a specific character appears.
- .find("word"): Returns the index (position) of the first occurrence of a word.

4. Variable Overriding

In Python, variables are dynamic. If i assign a new value to an existing variable name, the old value is replaced (overridden).

- *Example:* If number = 10 and then number = 30, the final value stored is 30.

—-7th Class—-

Python String Methods and Operations

1. Fundamental String Operations

Python strings are indexed sequences of characters where indexing starts at 0 and negative indexing starts from the end at -1.

Operation	**Description**	**Example (x = "hello world")**
len(x)	Returns the total number of characters in the string.	11
Indexing [i]	Accesses a character at a specific position.	x[4] → 'o'
Negative Indexing [-i]	Accesses characters starting from the end.	x[-1] → 'd'

String Slicing

i-x[0:5] / x[:5]: Extracts characters from index 0 up to (but not including) index 5 (example: "hello").

ii-x[6:]: Extracts characters from index 6 to the very end (example:"world").

2.String Modification Methods

1-.replace(): Replaces a specific part of the string with new text.

- `"rocky".replace("r", "p")` → output: `'pocky'`

2-.strip(): Removes extra spaces from both the start and the end.

- `" hello ".strip()` → output: `'hello'`

3.lstrip(): Removes spaces only from the left side.

- `" hello ".lstrip()` → output: `'hello '`

4.rstrip(): Removes spaces only from the right side.

- `" hello ".rstrip()` → output: `' hello'`

5.upper(): Converts all letters to uppercase.

- `"python".upper()` → output: `'PYTHON'`

6.lower(): Converts all letters to lowercase.

- `"PYTHON".lower()` → output: `'python'`

7.title(): Capitalizes the first letter of every word.

- `"hello world".title()` → output: `'Hello World'`

8.split(): Splits the string into a list based on a separator.

- `"a,b,c".split(",")` → output: `['a', 'b', 'c']`

9.capitalize(): Capitalizes only the first letter of the entire string.

- `"hello".capitalize()` → output: `'Hello'`

Methods:

1-.startswith(): Checks if the string starts with a specific value.

- `"apple".startswith("a")` → output: `True`

2-.endswith(): Checks if the string ends with a specific value.

- `"apple".endswith("e")` → output: `True`

3-.find(): Returns the index position of the first occurrence of a value.

- `"hello".find("e")` → output: 1

4-.count(): Counts how many times a value appears in the string.

- `"banana".count("a")` → output: 3

5-.isdigit(): Checks if all characters in the string are numbers.

- `"123".isdigit()` → output: `True`

6-.isalpha(): Checks if all characters in the string are letters.

- `"hello".isalpha()` → output: `True`

4. String Validation

Basically,return True or False.

- .isalpha(): True if all characters are alphabetic (A-Z).
- .isdigit(): True if all characters are digits (0-9).

- .isalnum(): True if all characters are alphanumeric (letters or numbers).
- .isspace(): True if the string consists only of whitespace.
- .istitle(): True if the string follows title case rules.
- .islower(): True if all characters are lowercase.

5. Input and Output Formatting

User Input

1-name = input("Enter your name: ")

2-age = int(input("Enter your age: "))

Printing Techniques

1. Standard Print: print("Name:", name, "Age:", age) or,
2. F-String: print(f"Your name is {name} and your age is {age}").

—8th Class—

if Statement: Executes the code block only if the condition is **True**.

- `if age >= 18:` → **Result:** Checks if age is 18 or more.

elif (else if): Used to check multiple conditions one after another if the previous ones were False.

- `elif score >= 80:` → **Result:** Checks this only if the first `if` failed.

else: Executes only if none of the above conditions are met.

- `else:` → **Result:** The final option and if any of the condition fail then else condition will work

—9th class—

The Python ternary operator evaluates a condition and returns a specific value in a single line

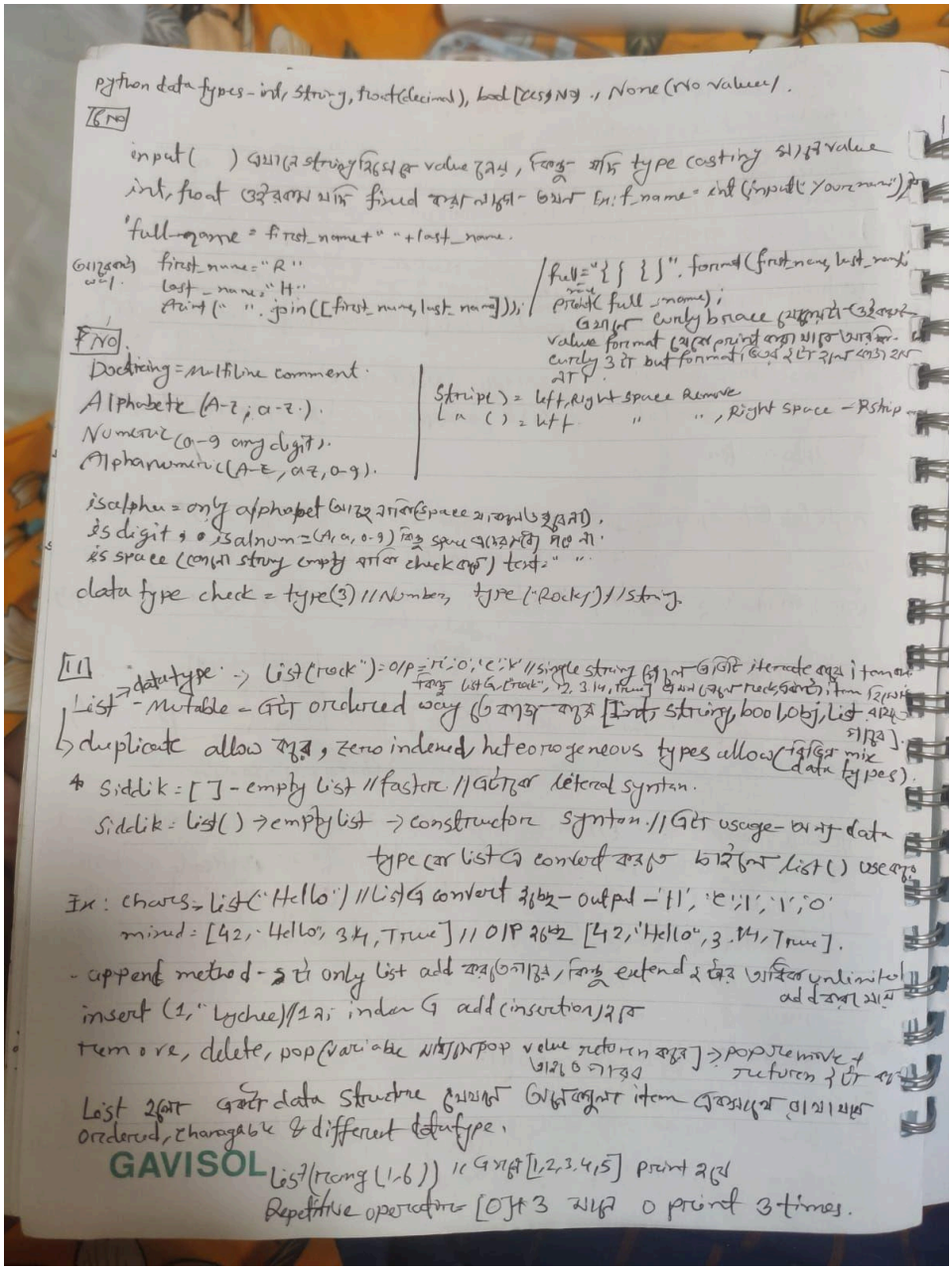
Ternery operator = `[Value_If_True] if [Condition] else [Value_If_False].`

```
status = "Pass" if marks >= 40 else "Fail"
```

i-Logic: If marks are 40 or more, status is "Pass"; otherwise, it is "Fail".

ii-Output: "Pass" (if marks = 50)

10 th 11 th no class



Accessing Elements: fruits = ["apple", "banana", "cherry", "date", "pineapple"]

index = 0 1 2 3 4

neg index -5 -4 -3 -2 -1

print(fruits[1]) // banana, print(fruits[-2]) // date

Slicing → fruits[2:4] মানে 2 থেকে 4 এর আগে পর্যন্ত 3 পর্যন্ত print.

fruits[:3] মানে 0-3 index.

fruits[3:] " 3 থেকে rest index.

fruits[0:4:2] = এখানে 0 শুরু index, 4 last index আগে পর্যন্ত, ~~এখানে~~ ~~step print~~ এর পর 2 নং index

এখানে 2 বলতে কয়টা print করা হবে তা বোঝায় 3 নং index বাদে print করবে এর পর 5 নং

Ex: উপরের ex: অনুসারে, apple, cherry print করবে।

fruits[::2] মানে [শুরু / শেষ দেওয়া নাই তাই শুরু 0 index হতে n-1 পর্যন্ত

fruits[::2] সাথে 2 টা index উপরের মত করব

ইনডেক্স ধরে নিয়ে আর skip করে

fruits(fruits[::-1] negative ভাবে উল্টো করে print করবে - করব

Replacement value

* fruits[0:3] = ["Mango", "Lychee", "Dragon"]. // এখানে 0-2 নং index পর্যন্ত value গুলো..

print(fruits) // আগের টা মুছে নতুন value updated হয়ে assign হবে

* fruits[1:1] = ["Mango", "Lychee", "Dragon"] // 1নং index এ এই 3টা value পর পর যুক্ত হবে.

fruits[2:4] = [] // empty রাখব //delete এর process [2/3 নং indexdelete হবে

fruits[1] = ["Lychee"] // list হিসেবে insert হবে, কিন্তু ওই index replacement হবে

* del fruits[0] // 0 index value delete ; del fruits[1:3] // 1-2 নং index delete.

concatenation:

* fruits1 = ['a', 'b', 'c', 'd'], fruits2 = ['e', 'f', 'g']

fruits = fruits1 + fruits2 : print(fruits) // o-p → ['a'...'d', 'e', 'f', 'g']. সবগুলো.

* append fruits.append("Mango") // last এ যোগ হবে // 1টা argument বেশি হলে নেওয়া যাবে না

("Mango", "Banana") // possible হবে না কেননা 1টা argument.

* যদি 2টার বেশি add করতে চাই extend method use হবে, fruits.extend(["Mango", "Lychee"]).

* extend("Mango", "Lychee") characterwise আসবে কিন্তু ["Mango", "D"] item wise আসবে যদি [ ] এ রাখো

* fruits.insert(1, "Lychee") // 1 no index এ এই value pass করবে.

Remove:

fruits.remove("apple") // apple remove হয়ে যাবে duplicates থাকলে আগের টা শুধু

fruits_pop_return = fruits.pop() // pop এর কাজ শেষের value return করবে লাস্ট value pop করে, list argument থাকলে ~~টা delete হবে~~, last return pop item হবার জন্য pop_return variable use হয়েছে.

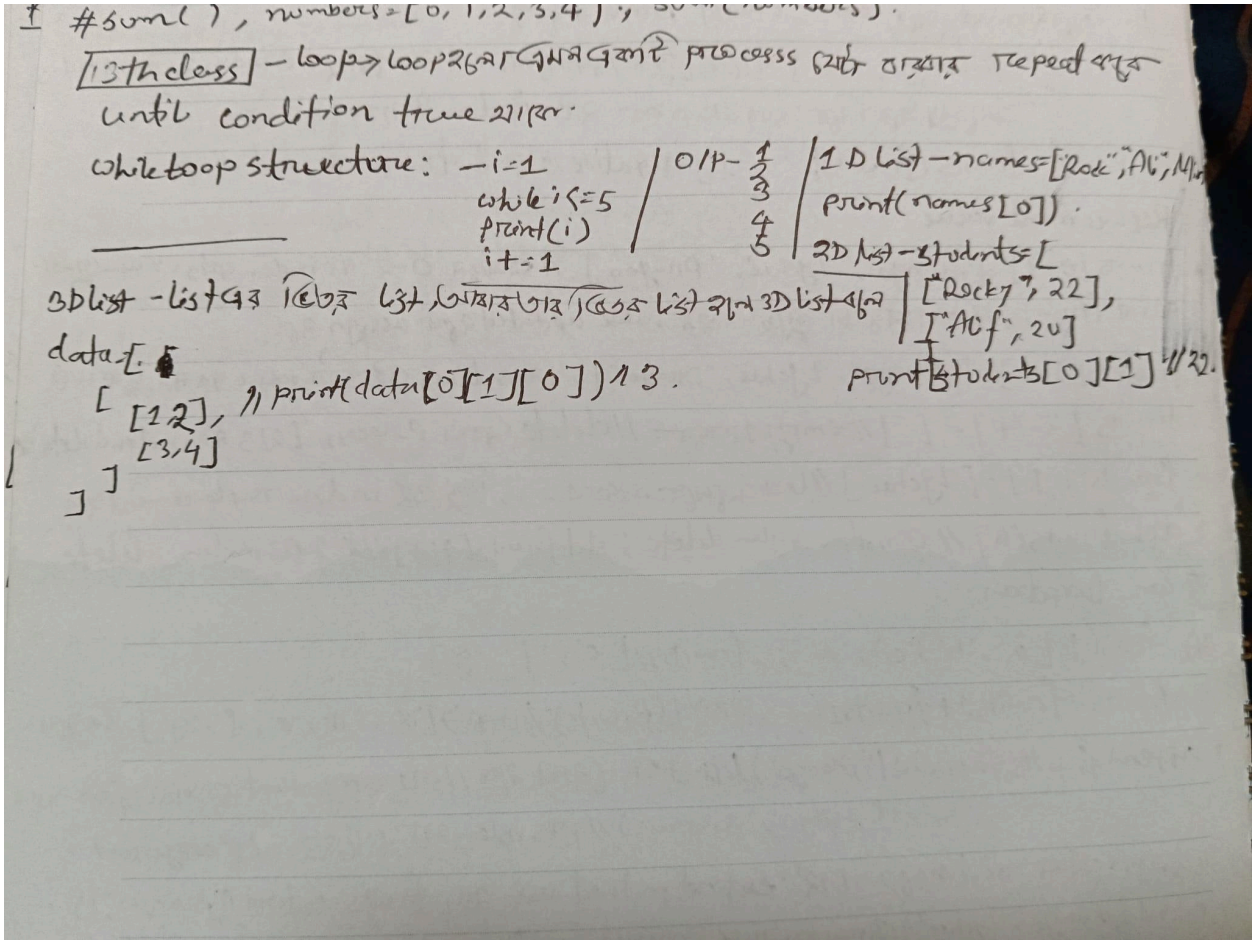
GAVISOL

যদি fruits.pop(2) // index wise pop করবে, যদি কিছু না থাকে তাহলে last value pop করবে

fruits.clear() // list empty করবে।

fruits.sort() // এটা alphabet অনুসারে ascii value. // fruits.sort(reverse = True) // reverse way তে sort হবে.

fruits.index() // index নং বলবে, fruits.count("apple") // apple কয়টা আছে count করে জানাবে



Till-24th no class—

3D list - list এর ভিতর list, আবার তার ভেতর list হলে 3D list বলে | ["ACf", 20]

```
data = [
  [
    [1,2],
    [3,4]
  ]
]
```

// print(data[0][1][0]) // 3.

print(students[0][1]) // 22.

print(f"...") এ f means formatted string.

18 For loop — iterate করার known sequence., while loop - Repeating until a conditions become false.

Ex: for i in range(5) // 5 এর আগে, 4 পর্যন্ত print | while x <= 5;

প্রতিটা item একএকে করে যাচ্ছে এটাই iteration.

range(stop), range(start, stop), range(start, stop, step).

Ex: Fruits = ["Mango", "Banana", "Strawberry"].

```
for i in Fruits
    print i
```

// এখানে লুপের শুরুতে python এই list এর উপর iter() fn চালায়, অর্থাৎ pointer, to track position. তারপর প্রতিবার loop ঘুরার সময় next() fn কল করে, প্রথমবার next() call করলে python পয়েন্টার apple এর i তে দেয়. ২য় বার কল করলে pointer এক ঘর ডানে সরে banana তে। iter() এর পরে পাওয়া Next fn দিয়ে list এর মধ্যে চলাফেরা যায়।

for loop: নির্দিষ্ট সংখ্যা বার ঘুরে

while: জানি না কতবার ঘুরবে কিন্তু শর্ত সত্যি থাকা পর্যন্ত চলবে.

numbers = list(range(1,11)) [list() আগে empty তখন range(1,11) থেকে নতুন list থেকে সব value তুলে নাও।

index based.

```
for i in range(len(fruits)):
    prints(fruits[i]).
```

/ String:

```
for i in 'Rocky'
    print(i) // R O C K Y
```

Reversed করা যায়,

```
numbers = [1, 2, 3, 4, 5].
for n in reversed(numbers):
    print n // এখানে 5 থেকে print হবে
```

//

```
names = ["Bd", "AB", "EA"].
for name in sorted(name, reverse=True):
    print(name) // sorted && reverse both.
```

enumerate ২টি জিনিস একসাথে দেয় – index (i), value (j).

```
names = ["A", "C", "B"]
for i, j in enumerate(names):
    print(i, j)
//O-P: 1 0 A
       2 1 C
       3 2 B.
```

যদি comma দিয়ে start=1 দেই তখন indexing 1 থেকে শুরু হবে

// print() আগে print না হয় line একটা নিচে নেমে যাবে
Hello
print() // মানে ২ line নিচে
World

break: condition meet করলে থেমে যাওয়া –
continue: " " " skip করা। [Ex i==2 continue; [২ বাদে সব print হবে]

20 -class:

LIST comprehension: python এর সংক্ষিপ্ত উপায়ে list তৈরী করা পদ্ধতি

```
squares = [ ] // Traditional Loop.
for x in range(5).
    squares.append(x**2).
```

| squares = [x**2 for x in range(5)].
// এখানে range(5) থেকে x নিয়ে তারপর x সাথে ২ গুন করে list এর মধ্যে রাখবে

OR,

```
num = [1, 2, 3, 4, 5].
even = []
for x in num
    if x % 2 == 0:
        even.appen(x)
print(even)
```

| numbers = [1, 2, 3, 4, 5]
even = [x for x in num if x % 2 == 0]
↑ যেই append হবে ↑ Loop টি ↑ যেই condition টি থাকবে সেটা না থাকলে নাই

data structures.

(v.v.i) Data structures: ডেটা রাখার জন্য বিভিন্ন container, যেগুলোর নিজস্ব নিয়ম এবং বিশেষ সুবিধা আছে।
– the way of organizing & storing data so that it can be accessed & modified efficiently.

List: অনেক item একসাথে রাখে, fruits = ["apple", "banana", "mango"]. [list mutable] [List এ append করা যায়]

Tuple: List এর মত কিন্তু change করা যায়না, fruits = ("apple", "banana", "mango") [Tuple immutable] (first bracket use হয়) [Tuple এ append করা যায়না]

Set: Duplicate value থাকেনা, numbers = {1, 2, 3}.

Dictionary: key, value pair এ রাখে, student = {"name": "Rocky", "age": 25}

List-mutable, Tuple-immutable
use = [] . , use = ( ) .
যে order data রাখে, সেই order এ রাখবে
duplicate allow. duplicates allow
ordered ordered (index করে দেখা যায় তাই ordered হয়, কিন্তু set এ index নাই তাই unordered)

| count: num = [1, 2, 3, 1, 5, 6, 7]
num.count(1) // কয়বার 1 আছে বের করবে

Tuples are faster than list (কারণ Tuples কোনো কিছু change হয় না), tuple গুলোতে unique value থাকে না।

Set: unordered, unique item রাখে (যেগুলো duplicate থাকলেও remove করে দিবে), index থাকে না, order maintain করে না ~~dyna~~ - faster, mathmetic type কাজে ব্যবহার করা হয়, { } use করে।

চাইলে list থেকে set আবার tuple এ convert ও করা যায়।

set mutable, index নেই তবে .add দিয়ে my_set.add(100) এভাবে নতুন value দেয়া যায়।

empty: | list(), shortform [ ] | tuple(), shortform ( ) | set( ) shortform নাই

কিন্তু my_set = { } //এটাতে dictionary বুঝায়

[redacted] remove করতে হলে আগে delete করতে হবে না থাকলে error দিবে
discard("fruit") " " " " " " error দিবে না skip করে চলে যাবে।

Set এর মধ্যে update নিয়মঃ fruits.update(["mango", "grape"]).
set এ joining set এর জন্য: fruits add ("cherry").

A - B = difference - in A but Not in B - [A তে যেটা B এর সাথে match করবে সেগুলো বাদ যাবে]: A হতে, বাকিগুলো Answer.

A ^ B = symmetric diff - A & B এর মধ্যে মিল বাদ দিয়ে সবগুলো একসাথে রাখা

A | B → union, A & B → intersection.

Dictionary: ordered, changeable, syntax {key : value}.

* .get("score", N/A) যদি key না থাকে তাহলে N/A print করবে

Ex: student = {"name": "Aiko", "age": 22, "grade": "A"}
(key: "name", value: "Aiko")

removed = student.pop("grade") → key wise remove,

del. student ["age"] → Full age key remove.

student.popitem() // Last key value remove.

student.clear sob clear.

set এ add, update করা যায়, কিন্তু -
Colors = {"red", "green"}.
colors.add ("blue").
color.remove ("red").
কিন্তু colors[0] = "pink" (index দিয়ে change করা যায় না).
existing item modify করা যায় না।

Comparison:

	list	Tuple	set	dictionary
ordered	Yes	Yes	No	3.7+ version হতে
changeable	Yes	No	Yes	Yes.
Allow duplicate	Yes	Yes	Auto remove	value allow keys not allow.
Access	index	index	not index	["key"].
empty	[ ]	( )	set( )	{ }
	O(n) slow	O(n) slow	O(1) fast	O(1) fast.
memory	medium	smallest	larger	largest

Set mutable কিন্তু একটা জিনিস আছে add remove করা যায় না সেটা frozenset.

* sorted() সবসময় list return করে

Set হচ্ছে

*args → Multiple positional arguments গ্রহণ করে এবং tuple হয়ে

**kwargs → Multiple keyword arguments গ্রহণ করে এবং dictionary তে রাখে
keyword arguments.

GAVISOL

* round এর কাজ: দশমিকের পর ২ ঘর হলে / বেশি ২ ঘর round করবে,
Ex: (56,2) = 56, (56.0, 2) = 56.0, (56.6789, 2) = 56.68.

* .2f হচ্ছে ২ ঘর অবশ্যই হবে কিছু থাকুক বা না থাকুক Ex: 56.6789 = 56.68, 56.0 = 56.00.

Lamda function: Anonymous function বলা হয় [def square এটা হচ্ছে named function].
↳ এর নাম নাই,
Why use: usually একই কাজ বার বার করলে তখন named fn use করি, কিন্তু যেটা frequently না হয় map(), filter(), sorted() use হয় (বেশি) use হবে না তখন small fn brief এ কাজ করা যায়,

Equivalent form: def square(x): return x**2; // named fn.
square lambda = lambda (x): x**2 ← যেটা return করতাম সেটা
keyword ↳ argument (normal fn এর)

Map: map() - এটি কোনো list / অন্য iterable এর element এর উপর fn প্রয়োগ করে
map(function, iterable) → যেমন list, tuple, range, dic, zip, map, filter etc.
↳ কোন fn চালাতে চাই Ex: square, lambda, len, max, sum [যেগুলো built in].

Filter: শুধু শর্ত পূরণ element রাখে, যেখানে fn true/false return করে, এখানে element পরিবর্তন হয় না true/false গুলো return করে, map এ হচ্ছে element change করে

Ex, num = [1, 2, 3, 4, 5].
result = list (filter(lambda x: x % 2 == 0, num)) print # result: [2, 4].

Sort: sorted (iterable, key = None/len(), reverse = True/False) iterable = list/tuple/dictionary etc.
Key: হচ্ছে কিসের ভিত্তিতে sort করবে
sorted (words, key = len) // key এ len আছে মানে string length অনুসারে sort হবে.
sorted (words, key = lambda x: x[-1]) = শেষ character অনুযায়ী.

- textwrap fill (text, 4) // এটি string কে 4 টি word এ ভাগ করে then new line
↳ wraplength
textwrap.wrap(" , 4) // এটি list এ দেখায়, textwrap dedent (text) // এটি indentation space সরায় মানে অতিরিক্ত space.
join — ("\n").join(new_list) - মানে হচ্ছে list এর প্রত্যেক value এর পর new line হবে.